

© Akshat Paul and Abhishek Nalwaya 2019

Akshat Paul and Abhishek Nalwaya, *React Native for Mobile Development*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-4454-8_7

7. Native Bridging in React Native

Akshat Paul¹ and Abhishek Nalwaya²

(1) Gurgaon, Haryana, India

(2) Jaipur, Rajasthan, India

Learn the rules like a pro, so you can break them like an artist.

—Pablo Picasso

So far, you have learned how to build applications using modules and APIs available out of the box with React Native. There are times, however, when an application has to access a native iOS or Android API and its corresponding React Native module is not yet available. Perhaps you will have to reuse some existing bespoke Swift, Kotlin, or Objective-C code with your React Native application. In such scenarios we create something called Native modules, which allow us to write code in the native language of a platform. This chapter covers the following topics:

- What is Native Bridge
- Prerequisite for Native Bridge
- Native Bridge for iOS
- Native Bridge for Android

The concept of Native modules is a bit advanced, but in our experience every production-quality application at some point requires you to delve into a little bit of native programming. Therefore, we consider this an essential skill to know in your journey to becoming a master in React Native.

Native Bridge

To better understand Native modules, we will create a `Counter` example in Swift for iOS and Java in Android, and this will be used in our React app. This will be a cross-platform example, so the same React code will work in both iOS and Android.

Because many readers of this book might not have worked in Swift or Java, we have tried to keep the use of both these languages very basic, so it should be easily understandable.

Prerequisites for the Example

Because we are writing some code in Native, you should have the following development setup installed on your computer.

- Xcode for running the app for iOS

- Android Studio for running the app for Android
- React Native

We will first create a React Native app with the React Native CLI.

We could also use the Expo CLI, but then we would have to eject it to build a Native bridge.

```
$ react-native init CounterNativeModuleApp  
$ cd CounterNativeModuleApp
```

This will create the basic structure of the React Native app. It also contains two folders, `ios` and `android`, which have native code in Objective-C and Java, respectively. We first learn about bridging in iOS, and then use same repo to build for Android.

iOS Native Bridge

We will create a `Counter` class in Swift, which will have a static class variable `count` and two methods: one for incrementing the count and the other for getting the count value. We will then access this Swift class from JavaScript. Let's start by opening the `CounterNativeModuleApp.xcodeproj` file in the `ios` folder. It should open Xcode with your iOS code.

Create a new file by going to File ► New ► File and selecting Swift, as shown in Figure [7-1](#).

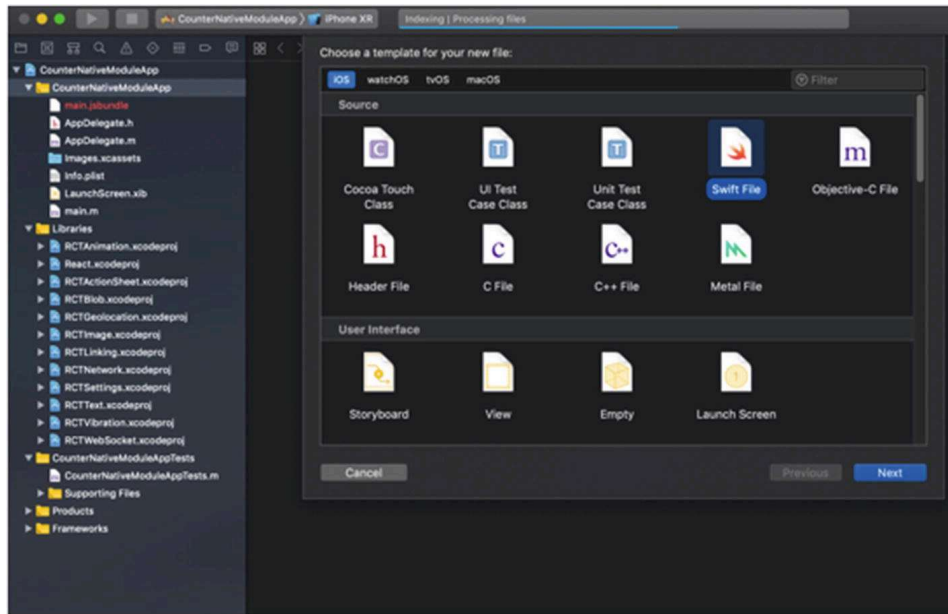


Figure 7-1 Creating a new file in Swift

Now give the file the name `Counter` and remember to select `CounterNativeModuleApp` for the Group setting, as shown in [Figure 7-2](#).

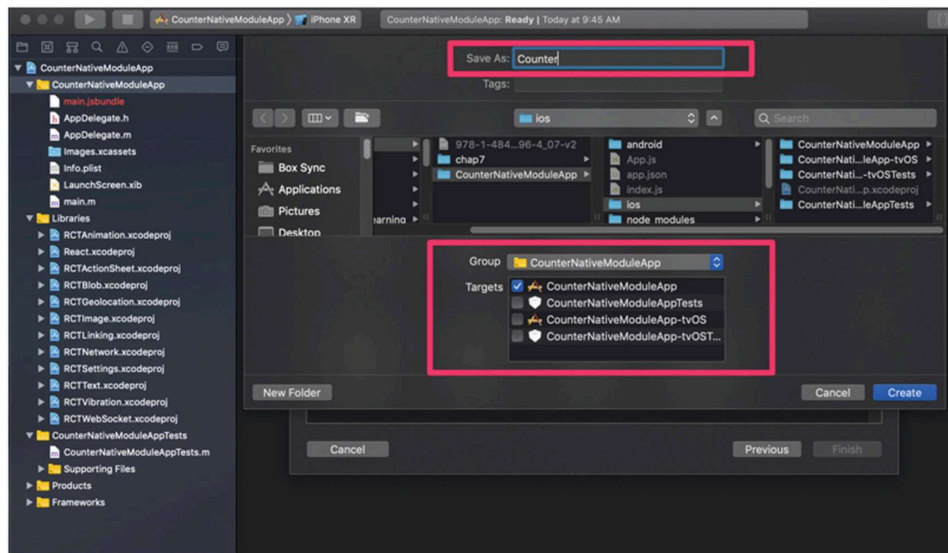


Figure 7-2 Selecting proper group in Xcode

As we are writing code in Swift and the repo, which is generated in Objective-C, we need a bridge to communicate between them. Click Create Bridging Header (Figure [7-3](#)).

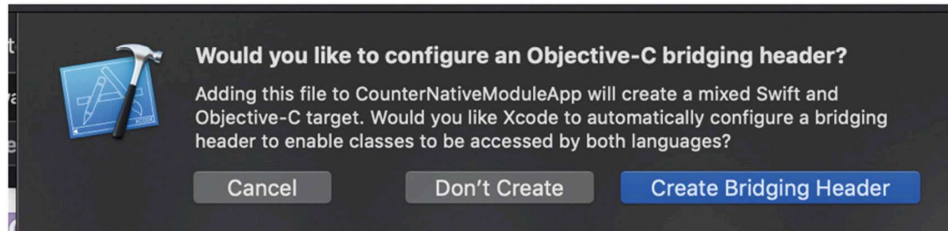


Figure 7-3 Creating a bridging header

We can see that two files, `Counter.swift` and `CounterNativeModuleApp-Bridging-Header.h`, are created by Xcode.

`Counter.swift` is where we will write our `Counter` class and `CounterNativeModuleApp-Bridging-Header.h` will have header details. Remember that in a project we have only one bridging header file, so if we add new files, we can reuse this file. Update the following code in the `CounterNativeModuleApp-Bridging-Header.h` file:

```
#import "React/RCTBridgeModule.h"
```

Now let's add a Swift class:

```
import Foundation
@objc(Counter)
class Counter: NSObject {
    @objc
    static var count = 0
    @objc
    func increment() {
        Counter.count += 1
        print("count is \(Counter.count)")
    }
}
```

In the preceding code we have created class `Counter`, which is inherited from `NSObject`. The root class of most Objective-C class hierarchies is `NSObject`, from which subclasses inherit a basic interface to the runtime system and the ability to behave as Objective-C objects.

You can see that we have used `@objc` before a function and class. This will make that class, function, or object available to Objective-C

NoteThe `@objc` attribute makes your Swift API available in Objective-C and the Objective-C runtime.

Now create a new file from File ► New ► File and select Objective-C File. Name the file `Counter` (Figure [7-4](#)).

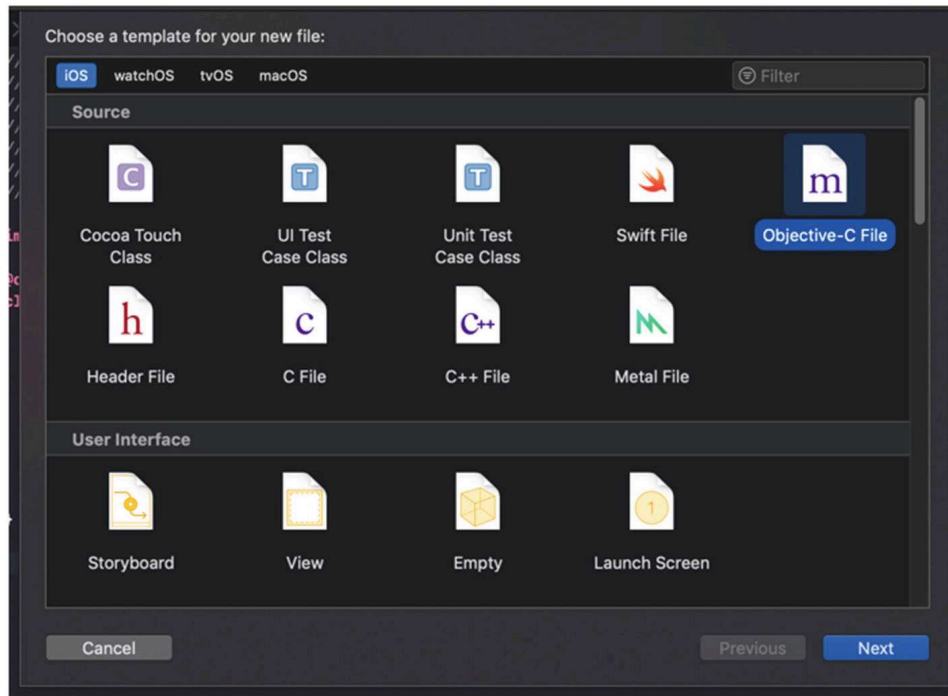


Figure 7-4 Creating an Objective-C file

This will create a file `Counter.m`, which will expose the Swift class to React Native:

```
#import "React/RCTBridgeModule.h"
@interface RCT_EXTERN_MODULE(Counter, NSObject)
RCT_EXTERN_METHOD(increment)
@end
```

React Native will not expose any function of `Counter` to React JavaScript unless explicitly done. To do so we can use the `RCT_EXPORT_METHOD()` macro. We therefore have to expose the `Counter` class and `increment` the method to our JavaScript code. Because the Swift object is converted to JSON, there is a type

conversion. `RCT_EXPORT_METHOD` supports all standard JSON object types:

- `string` (`NSString`)
- `number` (`NSInteger`, `float`, `double`, `CGFloat`, `NSNumber`)
- `boolean` (`BOOL`, `NSNumber`)
- `array` (`NSArray`) of any types from this list
- `object` (`NSDictionary`) with string keys and values of any type from this list
- `function` (`RCTResponseSenderBlock`)

Now let's update the JavaScript code and access this `Counter` class from our React component. To do so, open `App.js` and update it with the following code:

```
import React, {Component} from 'react';
import {StyleSheet, Text, View, NativeModules, Button} from 'react-native';
export default class App extends Component {
  increment = () => {
    NativeModules.Counter.increment();
  }
  render() {
    return (
      <View style={styles.container}>
        <Button
          onPress={this.increment}
          title="Increment"
          color="#841584">
```



```
        />
      </View>
    );
  }
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#F5FCFF',
  }
});
```

We need to import `NativeModule` from `react-native`. The `Counter` method `increment` can be accessed using `NativeModules.Counter.increment()`. We have created a `Button` and clicking on that button calls the `increment` method.

Now let's run the app from Xcode by pressing `Command + R`. Make sure React Native code is running. If it is not, then run `npm start`.

We can see an Increment button and a warning message at the bottom as shown in Figure [7-5](#). For now, ignore the warning message. We will talk about that later in the chapter.

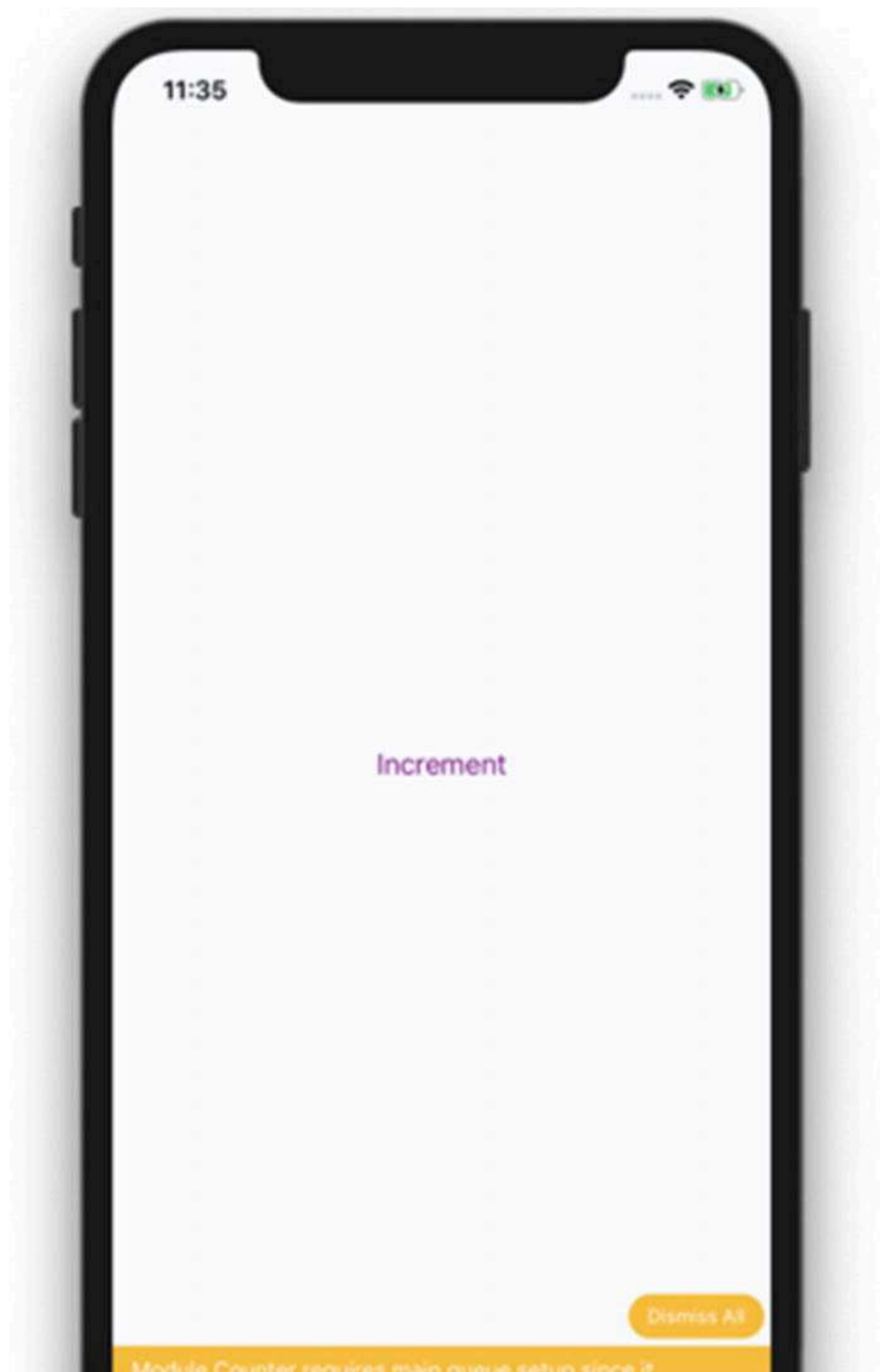




Figure 7-5 App running in a simulator

Now open the Xcode and check the console log. Try clicking the Increment button three times and you will see the output shown in the logs, as displayed in Figure 7-6.

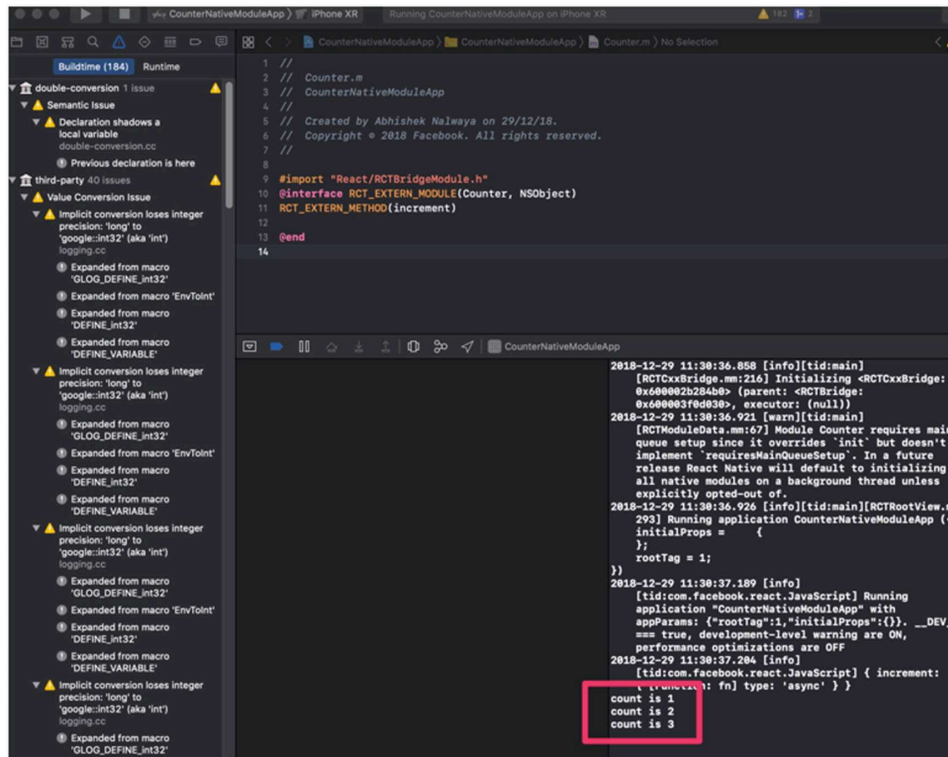


Figure 7-6 Increment displayed in console log

We can see that we have called a Swift class method from a JavaScript React component.

Note Remember, if you change any code in iOS Swift or Objective-C or Android Java, you need to rebuild the project. Only then will changes be reflected.

Now let's fix the warning shown at the bottom of the simulator and in the browser console:

Module Counter requires main queue setup since it overrides 'init' but doesn't implement 'requiresMainQueueSetup'. In a future release React Native will default to initializing all native modules on a background thread unless explicitly opted-out of.

To understand that better, let's understand the thread React Native runs on:

- *Main thread:* Where UIKit works.
- *Shadow queue:* Where the layout happens.
- *JavaScript thread:* Where your JavaScript code is actually running.
- Every native module has its own GCD (Grand Central Dispatch) Queue unless it specifies otherwise.

Now because this Native module will run on a different thread and our main thread is dependent on it, it is showing this warning. To make this code to run on `MainQueue`, open `Counter.swift` and add the following function:

```
@objc
static func requiresMainQueueSetup() -> Bool {
    return true
}
```

Now run the app again. Remember that because we have changed the Swift class, we need to rebuild the code. You will see the app running without the warning now, as shown in Figure [7-7](#).





Figure 7-7 Application running without warning

Now let's add the count value to our React screen. To do so we will add the `getCount` function to `counter.swift` and call that method from JavaScript code. We will create this method as a callback.

Note React Native Bridge is asynchronous, so the only way to pass a result to JavaScript is by using callbacks or emitting events.

Open `counter.swift` and add the `getCount` method :

```
import Foundation
@objc(Counter)
class Counter: NSObject {
    @objc
    static var count = 0
    @objc
    func increment() {
        Counter.count += 1
        print("count is \(Counter.count)")
    }
}
```

```

}
@objc
func getCount(_ callback: RCTResponseSenderBlock) {
    callback([NSNull(), Counter.count])
}
@objc
static func requiresMainQueueSetup() -> Bool {
    return true
}

```

The `getCount()` method receives a callback parameter that we will pass from your JavaScript code. We have called `callback` with an array of values, which will be exposed in JavaScript. We have passed `NSNull()` as the first element, which we consider an error in callback.

We need to expose this method to `counter.m`:

```

#import "React/RCTBridgeModule.h"
@interface RCT_EXTERN_MODULE(Counter, NSObject)
    RCT_EXTERN_METHOD(increment)

    RCT_EXTERN_METHOD(getCount: (RCTResponseSenderBlock)callback)

@end

```

Let's update the React code to take the count from the `getCount` method that we just created. Update `App.js` with following code:


```
import React, {Component} from 'react';
import {StyleSheet, Text, View, NativeModules, Button} from 'react-native';
export default class App extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
    this.updateCount();
  }
  increment = () => {
    NativeModules.Counter.increment();
    this.updateCount();
  }
  updateCount = () => {
    NativeModules.Counter.getCount( (error, count)=>{
      this.setState({ count: count});
    })
  }
  render() {
    return (
      <View style={styles.container}>
        <Text>Counter from Native Code:</Text>
        <Text>{this.state.count}</Text>
        <Button
          onPress={this.increment}
          title="Increment"
          color="#841584"
        />
      </View>
    );
  }
}
```

```
}  
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    justifyContent: 'center',  
    alignItems: 'center',  
    backgroundColor: '#F5FCFF',  
  }  
});
```

Rebuild the source code and run the app. You can then see the counter value and when you click Increment, it will increase the count as show, in Figure [7-8](#).



Figure 7-8 Application demo in simulator

Try to refresh the page by pressing Command + R. The count value will be the same and does not reset to 0. If you rebuild the code, however, then the value will be reset to 0.

Native Bridge for Android

In this section we will make the same JavaScript code work with Android. This time we will create a `Counter` class in java

and expose the same functions, `increment` and `getCount`, to Javascript.

Open Android Studio (Figure 7-9) and select Open an existing Android Studio project, and then select the `android` folder inside our `CounterNativeModuleApp`.

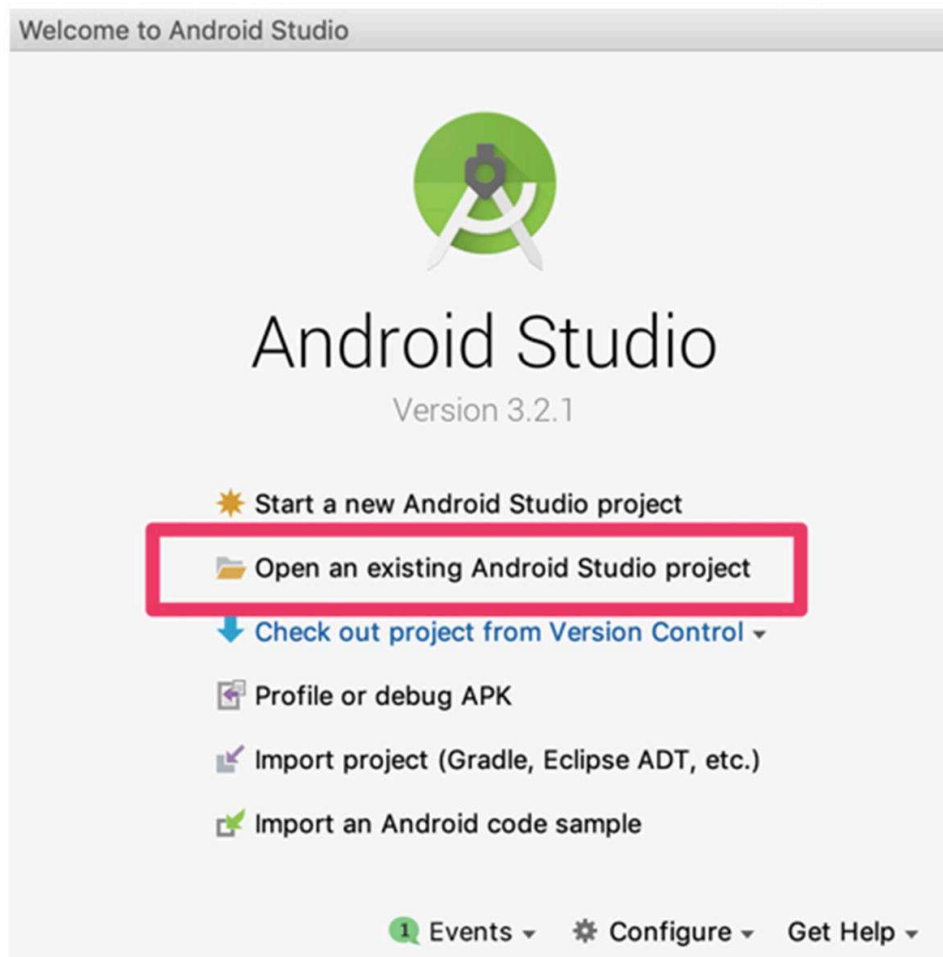


Figure 7-9 Open the React Native app in Android Studio

Once the project is opened and it has downloaded all gradle dependency (gradle is the dependency manager of Java), we will create a class `Counter`. Click Menu ► File ► New ► Java Class. Name the file `Counter` and then click OK (Figure 7-10).

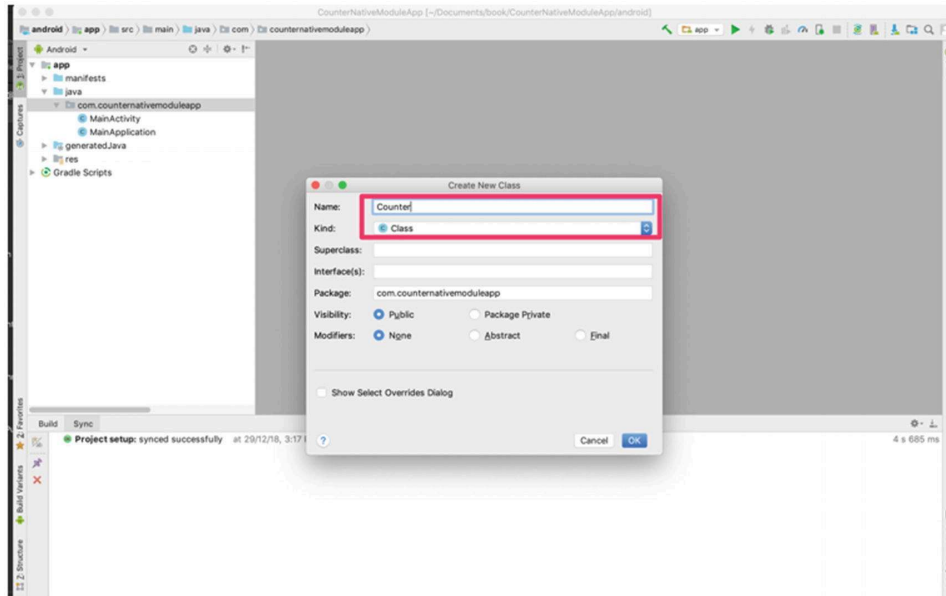


Figure 7-10 Creating a Counter class

Add the following code in `Counter.java` file:

```
package com.counternativemoduleapp;

import com.facebook.react.bridge.NativeModule;
import com.facebook.react.bridge.ReactApplicationContext;
import com.facebook.react.bridge.ReactContext;
import com.facebook.react.bridge.ReactContextBaseJavaModule;
import com.facebook.react.bridge.ReactMethod;
import com.facebook.react.bridge.Callback;
```

```
public class Counter extends ReactContextBaseJavaModule {
    private static Integer count = 0;
    public Counter(ReactApplicationContext reactContext) {
        super(reactContext);
    }
    @ReactMethod
    public void increment() {
        count++;
        System.out.println(count);
    }
    @ReactMethod
    public void getCount(
        Callback successCallback) {
        successCallback.invoke(null, count);
    }
    @Override
    public String getName() {
        return "Counter";
    }
}
```

We have created the Native module `Counter`, which is a Java class that is inherited from `ReactContextBaseJavaModule`.

`ReactContextBaseJavaModule` requires that the function `getName` is called; this is always implemented. The purpose of this method is to return the string name of the Native module, which represents this class in JavaScript. Here we will call this `Counter` so that we can access it through

`React.NativeModules.Counter` in JavaScript. Instead of `Counter`, you could also use a different name.

Not all functions are exposed to JavaScript. To expose a function to JavaScript, a Java method must be annotated using `@ReactMethod`. The return type of bridge methods is always `void`, so we create a function `increment` with `@ReactMethod` where we have to increase the value of the static variable `count` and then print the value in the console:

```
@ReactMethod
public void increment() {
    count++;
    System.out.println(count);
}
```

We have also created a `getCount` function that has `callback` as a parameter. It returns a `callback` and passes the value of `count`.

```
@ReactMethod
public void getCount(
    Callback successCallback) {
    successCallback.invoke(null, count);
}
```

The next step is to register the module, because if a module is not registered it will not be available from JavaScript. To create a file, click

Menu ► File ► New ► Java Class. Name the file

CustomCounterPackage and then click OK (Figure 7-11).

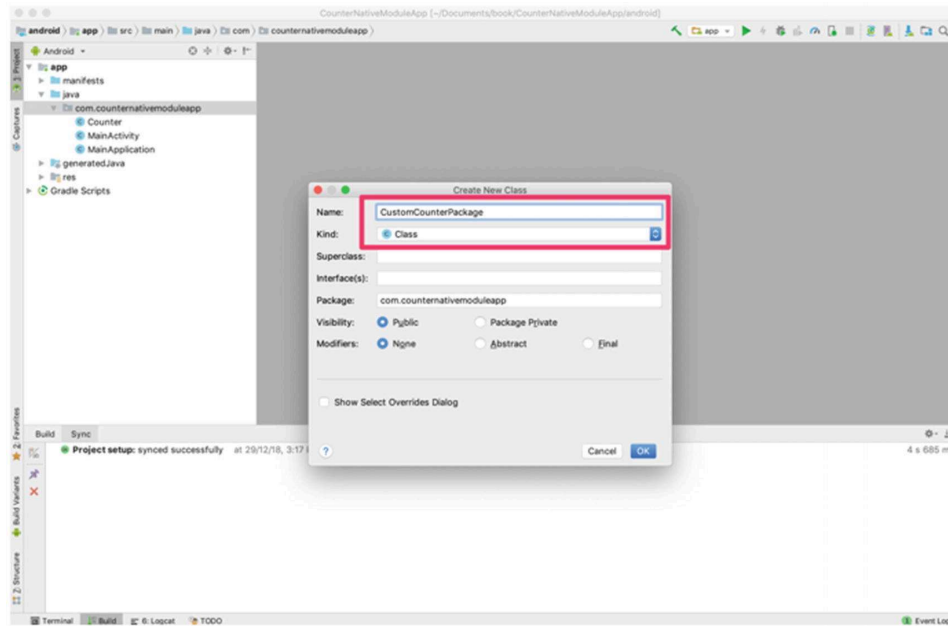


Figure 7-11 Creating a new Java class

Now add the following code in CustomCounterPackage:

```
package com.counternativemoduleapp;
import com.facebook.react.ReactPackage;
import com.facebook.react.bridge.NativeModule;
import com.facebook.react.bridge.ReactApplicationContext;
import com.facebook.react.uimanager.ViewManager;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
public class CustomCounterPackage implements ReactPackage {
    @Override
```



```
public List<ViewManager> createViewManagers(ReactApplicationContext reactContext) {  
    return Collections.emptyList();  
}  
@Override  
public List<NativeModule> createNativeModules(  
    ReactApplicationContext reactContext) {  
    List<NativeModule> modules = new ArrayList<>();  
    modules.add(new Counter(reactContext));  
    return modules;  
}  
}
```

We need to override the `createNativeModules` function and add the `Counter` object to the modules array. If this is not added there, it will not be available in JavaScript.

A `CustomCounterPackage` package needs to be provided in the `getPackages` method of the `MainApplication.java` file. This file exists in the `android` folder in your `react-native` application directory. Update the following code in `android/app/src/main/java/com/CounterNativeModuleApp/MainApplication.java`:

```
package com.counternativemoduleapp;  
  
import android.app.Application;  
import com.facebook.react.ReactApplication;  
import com.facebook.react.ReactNativeHost;  
import com.facebook.react.ReactPackage;
```

```
import com.facebook.react.shell.MainReactPackage;
import com.facebook.sololoader.SoLoader;
import java.util.Arrays;
import java.util.List;
import com.counaternativemoduleapp.CustomCounterPackage;
public class MainApplication extends Application implements ReactApplication {
    private final ReactNativeHost mReactNativeHost = new ReactNativeHost(this) {
        @Override
        public boolean getUseDeveloperSupport() {
            return BuildConfig.DEBUG;
        }
        @Override
        protected List<ReactPackage> getPackages() {
            return Arrays.<ReactPackage>asList(
                new MainReactPackage(),
                new CustomCounterPackage()
            );
        }
        @Override
        protected String getJSMainModuleName() {

            return "index";
        }
    };
    @Override
    public ReactNativeHost getReactNativeHost() {
        return mReactNativeHost;
    }
}
```

```

    }
    @Override
    public void onCreate() {
        super.onCreate();
        SoLoader.init(this, /* native exopackage */ false);
    }
}

```

We don't need to change any JavaScript code written in iOS, as we have exposed the same class name and function. If you skipped the iOS section earlier, you need to copy the React JavaScript code from `App.js`.

Now run the app through Android Studio or from `react-native run-android` (Figure 7-12).

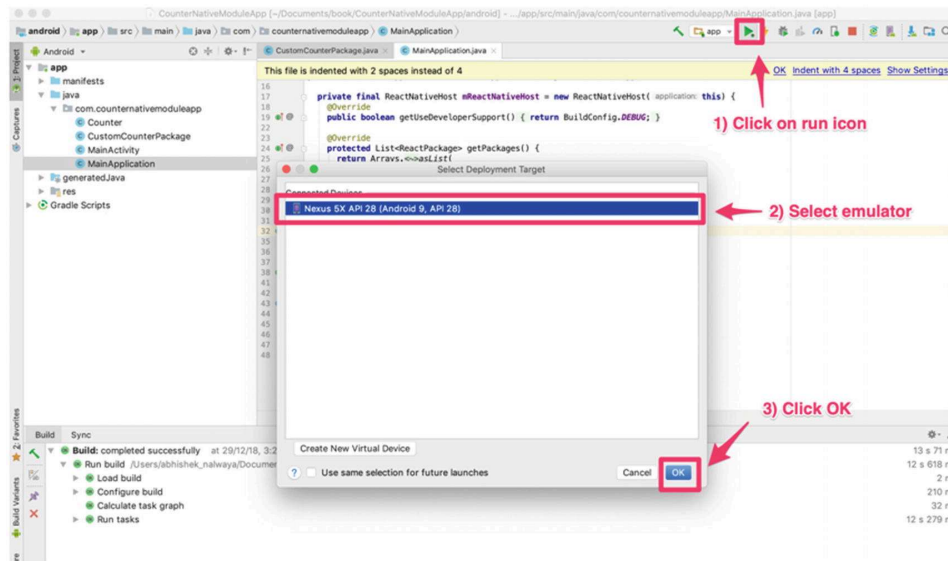


Figure 7-12 Run the application from Android Studio

This will launch the Android emulator with the app (Figure 7-13). If you don't find an emulator in the list, you need to download a few by clicking Create New Virtual Device.

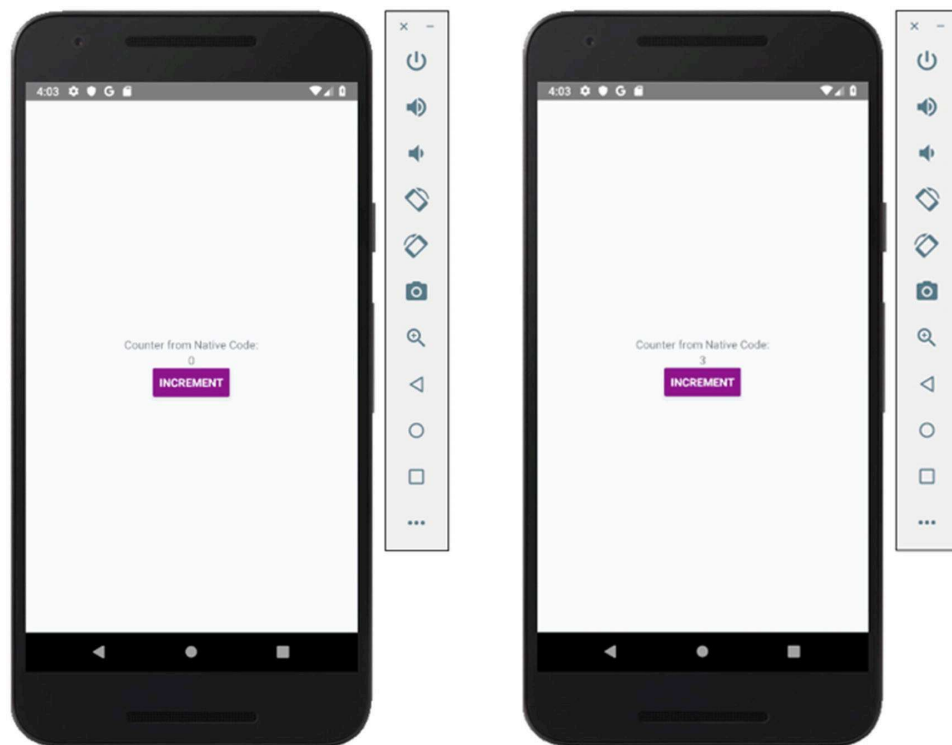


Figure 7-13 Application running in Android emulator

We can see the counter change when we click Increment and the JavaScript code is calling the Java code.

Summary

This chapter covered Native Bridge for both iOS and Android. You created a class in Swift and Java and through NativeBridge you were able to access these classes in JavaScript code.

In Chapter **8** you learn about testing in React Native, including type checking using Flow, using Jest with React Native, and understanding how to use snapshot testing.